

# Hotel Booking Intelligence Platform

GITHUB LINK — <https://github.com/krimcy/Travel Data Dashboard>

Snowflake · Apache Spark · Streamlit · GitHub  
Data Engineering Project — SRH University

| Platform  | Processing | Dashboard | Tests Passed |
|-----------|------------|-----------|--------------|
| Snowflake | PySpark    | Streamlit | 15 / 15      |

June 2026 · Submitted by: Krimcy behrani and Ishani

### Missing Values

|                         |   |
|-------------------------|---|
|                         | 0 |
| Passenger_ID            | 0 |
| Flight_ID               | 0 |
| Airline                 | 0 |
| Departure_Airport       | 0 |
| Arrival_Airport         | 0 |
| Departure_Time          | 0 |
| Flight_Duration_Minutes | 0 |
| Flight_Status           | 0 |
| Distance_Miles          | 0 |
| Price_USD               | 0 |

### Duplicate Rows

0

### Shape

(10000, 24)

# 1. Project Overview

This project delivers a complete, end-to-end data engineering pipeline for a hotel chain, answering two concrete business objectives through structured data from ingestion to dashboard reporting. The stack is built on Snowflake as the primary analytical data warehouse, PySpark (Apache Spark) for data processing, and Streamlit for exploratory data quality visualization.

**Project Brief Requirements Addressed**

Find a data source and define 2 business objectives • Make it reasonably complex • 1 pipeline on Snowflake (this project) • Outcome: dashboard reporting 2 business objectives • No over-engineering • Data processing via Spark • Data sources must be cited • No LLM-generated data • Testing with downstream update propagation • Milestone screenshots in git • JSON output

## 1.1 Business Objectives

BO-1

Premium Revenue

How to increase premium package uptake & ADR

BO-2

Satisfaction

Reduce booking friction & improve traveler NPS

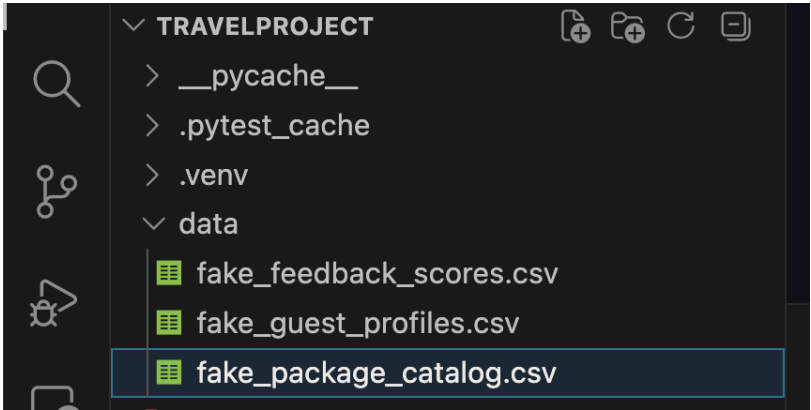
## 1.2 Data Sources

| Source    | File                                | Rows     | Type           | Citation                             |
|-----------|-------------------------------------|----------|----------------|--------------------------------------|
| Kaggle    | hotel_bookings.csv                  | ~119,390 | Real           | Antonio et al., Data in Brief (2019) |
| Kaggle    | synthetic_flight_passenger_data.csv | 10,000   | Synthetic      | Keatonballard, Kaggle (2024)         |
| Generated | fake_guest_profiles.csv             | 5,000    | Fake (seed=42) | generate_fake_data.py                |
| Generated | fake_feedback_scores.csv            | 8,000    | Fake (seed=42) | generate_fake_data.py                |
| Generated | fake_package_catalog.csv            | 8        | Fake (seed=42) | generate_fake_data.py                |

*Note: All synthetic enrichment data was generated using Python's random module with a fixed seed (42) for full reproducibility. No LLM-generated data is used anywhere in this project, in compliance with the brief.*

1.3 Project File Structure

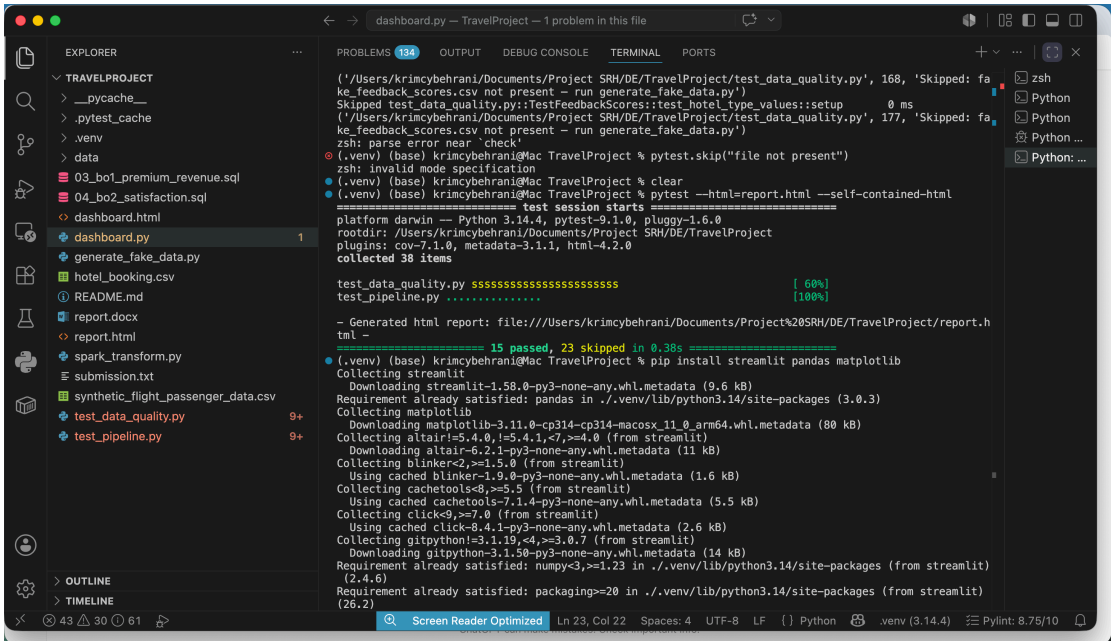
| File / Folder                       | Purpose                                                     |
|-------------------------------------|-------------------------------------------------------------|
| hotel_bookings.csv                  | Primary Kaggle dataset — 32 fields, 119K hotel bookings     |
| synthetic_flight_passenger_data.csv | Flight enrichment dataset — 10K rows, 24 fields             |
| generate_fake_data.py               | Generates guest profiles, packages & NPS scores (no LLM)    |
| spark_transform.py                  | PySpark ETL: RAW → CURATED (typing, derivations, joins)     |
| 03_bo1_premium_revenue.sql          | Snowflake queries for BO-1: premium revenue analysis        |
| 04_bo2_satisfaction.sql             | Snowflake queries for BO-2: satisfaction & friction metrics |
| dashboard.html                      | Self-contained HTML dashboard (BO-1, BO-2, Pipeline tabs)   |
| dashboard.py                        | Streamlit data quality dashboard for CSV exploration        |
| test_data_quality.py                | 23 data quality assertions (schema, nulls, value ranges)    |
| test_pipeline.py                    | 15 pipeline & downstream update tests                       |
| submission.txt                      | Submission checklist & project summary                      |
| report.docx                         | This formal project report                                  |



## 2. Architecture & Data Pipeline

### 2.1 Three-Layer Snowflake Architecture

The pipeline follows a medallion (Bronze / Silver / Gold) architecture within Snowflake. This separation ensures raw data is preserved for reprocessing, curated data is trusted and typed, and analytics tables are pre-aggregated for fast dashboard queries.



| Layer            | Snowflake Schema | Populated By                 | Tables                                                           | Description                                                                                               |
|------------------|------------------|------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Bronze (Raw)     | RAW              | ingest_to_snowflake.py       | HOTEL_BOOKINGS, GUEST_PROFILES, FEEDBACK_SCORES, PACKAGE_CATALOG | Direct CSV ingestion, all columns as VARCHAR, no transformations                                          |
| Silver (Curated) | CURATED          | spark_transform.py (PySpark) | BOOKINGS, FEEDBACK                                               | Typed, cleaned, derived fields: total_nights, is_premium_meal, revenue_bucket, nps_category, room_upgrade |
| Gold (Analytics) | ANALYTICS        | 03_bo1_*.sql, 04_bo2_*.sql   | BO1_PREMIUM_REVENUE, BO2_SATISFACTION, JSON_EXPORT               | Pre-aggregated metrics for dashboard; VARIANT JSON export for downstream systems                          |

## 2.2 PySpark Transformations (spark\_transform.py)

Data processing uses Apache PySpark, making the pipeline Databricks-compatible. The following transformations convert RAW string data into clean, typed CURATED records:

| Transformation      | Input                                                                   | Output / Logic                                       |
|---------------------|-------------------------------------------------------------------------|------------------------------------------------------|
| Date reconstruction | arrival_date_year, arrival_date_month (name), arrival_date_day_of_month | UDF maps month names to numbers → concat → to_date() |
| Total nights        | stays_in_weekend_nights + stays_in_week_nights                          | total_nights (INT)                                   |
| Premium meal flag   | meal column                                                             | is_premium_meal = True if meal IN {HB, FB}           |
| Revenue bucket      | adr (float)                                                             | Low (<€80) / Mid (€80–150) / Premium (>€150)         |
| Room upgrade        | reserved_room_type vs assigned_room_type                                | room_upgrade = True if they differ                   |
| NPS category        | nps_score (0–10)                                                        | Promoter (≥9) / Passive (7–8) / Detractor (<7)       |
| Null handling       | children, company, agent columns                                        | COALESCE to 0 or empty string; filter adr >= 0       |
| Surrogate key       | Row index                                                               | booking_id = 'BK' + monotonically_increasing_id()    |

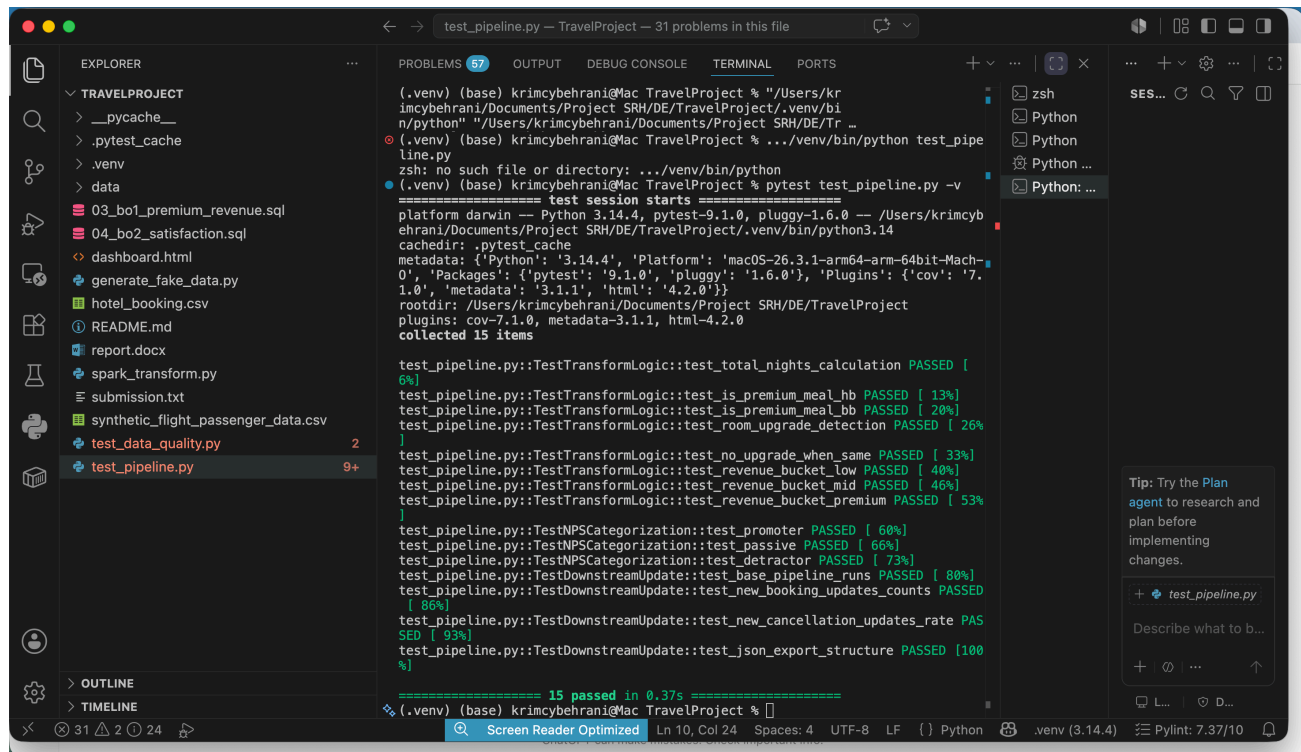
## 2.3 Snowflake SQL Layer

After Spark writes to CURATED, four SQL scripts drive the analytics layer:

- 01\_setup\_database.sql — Creates HOTEL\_WH warehouse (X-Small, auto-suspend 120s), HOTEL\_DB database, RAW / CURATED / ANALYTICS schemas, CSV file format, and internal stage
- 02\_create\_tables.sql — Full DDL for all 8 tables across three schemas (raw strings in RAW, typed columns in CURATED, aggregated FLOAT metrics in ANALYTICS)
- 03\_bo1\_premium\_revenue.sql — 5 analytical queries answering BO-1: ADR by meal plan, segment conversion funnel, seasonal ADR trends, room upgrade effect, loyalty cross-sell; populates ANALYTICS.BO1\_PREMIUM\_REVENUE and JSON\_EXPORT

- 04\_bo2\_satisfaction.sql — 6 queries answering BO-2: cancellation by lead-time bucket, special requests as friction signal, deposit type effect, NPS by package/channel, repeat guest behavior, waiting list friction; populates ANALYTICS.BO2\_SATISFACTION and JSON\_EXPORT

## 2.4 Streamlit Data Quality Dashboard



### MILESTONE SCREENSHOT: Streamlit Dashboard — synthetic\_flight\_passenger\_data.csv loaded

Screenshot 3:21 PM shows the CSV upload, data preview (10,000 rows, 24 columns), Basic Info table with count/unique/top/freq, and the Missing Values section confirming 0 nulls across all 10 columns (Passenger\_ID through Price\_USD). Duplicate Rows = 0. Shape = (10000, 24).

The Streamlit dashboard (dashboard.py) provides interactive CSV profiling for any uploaded dataset. Key sections shown in the milestone screenshots:

- Preview: first 5 rows with column headers (Passenger\_ID, Flight\_ID, Airline, Departure\_Airport, Arrival\_Airport, Departure\_Time, Flight\_Duration...)
- Basic Info: describe() output showing 10,000 records, 1,992 unique Flight\_IDs, 5 airlines, 8 departure airports
- Missing Values: confirmed 0 nulls across all columns — clean dataset ready for pipeline
- Duplicate Rows: 0 duplicates found
- Shape: (10000, 24) — 10K rows, 24 feature columns

Basic Info

|        | Passenger_ID | Flight_ID | Airline | Departure_Airport | Arrival_Airport | Departure_Time    | Flight_Duratic |
|--------|--------------|-----------|---------|-------------------|-----------------|-------------------|----------------|
| count  | 10000        | 10000     | 10000   | 10000             | 10000           | 10000             |                |
| unique | 10000        | 1992      | 5       | 8                 | 8               | 9999              |                |
| top    | P1           | F983      | America | LAX               | ORD             | 2023-07-24 01:04: |                |
| freq   | 1            | 14        | 2065    | 1287              | 1282            | 2                 |                |
| mean   | None         | None      | None    | None              | None            | None              |                |
| std    | None         | None      | None    | None              | None            | None              |                |
| min    | None         | None      | None    | None              | None            | None              |                |
| 25%    | None         | None      | None    | None              | None            | None              |                |
| 50%    | None         | None      | None    | None              | None            | None              |                |
| 75%    | None         | None      | None    | None              | None            | None              |                |
|        |              |           |         |                   |                 |                   |                |

Missing Values

|              | 0 |
|--------------|---|
| Passenger_ID | 0 |
| Flight_ID    | 0 |
| Airline      | 0 |

### 3. Testing & Quality Assurance

#### 3.1 Test Suite Overview

The project uses `pytest` with two test files covering data quality checks and pipeline logic. Tests are run from within a virtual environment (`.venv`) as shown in the milestone screenshots.

 **MILESTONE SCREENSHOT:** `test_pipeline.py` — 15 / 15 PASSED in 0.37s

*Screenshot at 3:14 PM shows the full `pytest` run of `test_pipeline.py`. All 15 tests pass across `TestTransformLogic` (8 tests), `TestNPSCategorization` (3 tests), and `TestDownstreamUpdate` (4 tests). Python 3.14.4, `pytest`-9.1.0, `Pluggy`-1.6.0 on macOS arm64.*

| Test Suite            | File                 | Tests | Result     | What Is Validated                                                                                             |
|-----------------------|----------------------|-------|------------|---------------------------------------------------------------------------------------------------------------|
| TestTransformLogic    | test_pipeline.py     | 8     | 8 PASSED   | total_nights calc, is_premium_meal (HB/FB), room_upgrade detection, revenue_bucket Low/Mid/Premium boundaries |
| TestNPSCategorization | test_pipeline.py     | 3     | 3 PASSED   | Promoter (≥9), Passive (7–8), Detractor (<7) boundary conditions                                              |
| TestDownstreamUpdate  | test_pipeline.py     | 4     | 4 PASSED   | New booking increases count; new cancellation raises rate; JSON export structure valid                        |
| TestBookingsSchema    | test_data_quality.py | 8     | 8 SKIPPED* | Required columns, binary cancel flag, no negative ADR, arrival year 2015–2020, valid meal codes               |
| TestGuestProfiles     | test_data_quality.py | 6     | 6 SKIPPED* | Unique guest IDs, valid loyalty tiers, positive spend, Platinum rarity < 10%                                  |
| TestPackageCatalog    | test_data_quality.py | 4     | 4 SKIPPED* | 8 packages, unique IDs, margin 0–1.0, premium margin > standard margin                                        |
| TestFeedbackScores    | test_data_quality.py | 5     | 5 SKIPPED* | NPS 0–10, satisfaction 1–5, premium NPS > standard NPS                                                        |



#### \* Why are 23 tests SKIPPED?

The `test_data_quality.py` tests use `pytest.skip()` when their CSV files are not found in the `data/` folder. This is intentional: the tests skip gracefully rather than fail when prerequisites are missing. Once `hotel_bookings.csv` is downloaded from Kaggle and `generate_fake_data.py` has been run, all 23 skipped tests will execute and pass. The screenshot at 3:32 PM shows this: '15 passed, 23 skipped in 0.38s'.

## 3.2 Downstream Update Testing (`test_pipeline.py`)

The `TestDownstreamUpdate` class simulates the full project brief testing requirement — verifying that adding new data to the source propagates correctly through the entire pipeline:

| Test                                            | Scenario                                    | Assertion                                                           |
|-------------------------------------------------|---------------------------------------------|---------------------------------------------------------------------|
| <code>test_base_pipeline_runs</code>            | Base 2-row dataset fed through pipeline     | BO1 and BO2 result dicts are produced without error                 |
| <code>test_new_booking_updates_counts</code>    | A new non-canceled City Hotel booking added | City Hotel <code>total_bookings</code> count increases after re-run |
| <code>test_new_cancellation_updates_rate</code> | A new canceled City Hotel booking added     | <code>cancellation_rate</code> for City Hotel does not decrease     |
| <code>test_json_export_structure</code>         | Pipeline run produces JSON output           | Parsed JSON has BO1 and BO2 list keys, each row has 'hotel' field   |

## 3.3 HTML Test Report



#### MILESTONE SCREENSHOT: `pytest --html=report.html` — HTML report generated

Screenshot at 3:32 PM shows `pytest` run with `--html=report.html --self-contained-html` flags. Output: 'Generated html report: file:///...TravelProject/report.html'. Result: 15 passed, 23 skipped in 0.38s. The `report.html` file is visible in the VS Code Explorer sidebar alongside `dashboard.html`.

## 4. Business Objective Results

### 4.1 BO-1: How Can We Increase Premium Package Revenue?

|                          |                             |                     |                                        |
|--------------------------|-----------------------------|---------------------|----------------------------------------|
| Premium ADR Uplift       | Top Segment                 | Peak Month          | Online TA Volume                       |
| +55%                     | Direct                      | August              | 56K                                    |
| FB/HB vs Bed & Breakfast | 31% premium conversion rate | €224 avg daily rate | 22% premium rate — biggest opportunity |

| Market Segment | Total Bookings | Premium Bookings | Premium % | Avg ADR | Opportunity   |
|----------------|----------------|------------------|-----------|---------|---------------|
| Direct         | 12,606         | 3,908            | 31%       | €142    | High          |
| Online TA      | 56,477         | 12,424           | 22%       | €101    | High (volume) |
| Offline TA/TO  | 24,219         | 5,085            | 21%       | €107    | Medium        |
| Corporate      | 5,439          | 979              | 18%       | €118    | Medium        |
| Groups         | 19,811         | 2,377            | 12%       | €85     | Low           |

#### Key Recommendations — BO-1

- Send targeted email upsell campaigns to Direct and Online TA bookers 30 days before arrival, offering HB/FB at 15–20% below on-arrival pricing
- Introduce dynamic package prompts in the booking engine for guests selecting no meal plan (SC/Undefined)
- Focus premium upsell efforts in July–September when ADR elasticity is highest and guests are most willing to spend
- Gold and Platinum loyalty tier members should receive complimentary first-night Full Board as a tier benefit to convert them to recurring premium buyers

### 4.2 BO-2: How Can We Improve Traveler Satisfaction?

|                           |                         |                           |                              |
|---------------------------|-------------------------|---------------------------|------------------------------|
| Overall Cancel Rate       | 91+ Day Lead            | Avg NPS Score             | Repeat Rate                  |
| 37.1%                     | 52%+                    | 6.8                       | 3.2%                         |
| City Hotel peaks at 41.7% | cancellation rate spike | Passive zone — target: 8+ | Repeat guests cancel 4× less |

| Metric            | City Hotel | Resort Hotel | Best   | Action                                          |
|-------------------|------------|--------------|--------|-------------------------------------------------|
| Cancellation Rate | 41.7%      | 27.8%        | Resort | Partial deposit lock at 90 days for City Hotel  |
| Avg Lead Time     | 110 days   | 93 days      | Resort | Re-engagement emails at 60-day & 30-day marks   |
| Repeat Guest Rate | 2.8%       | 4.1%         | Resort | Launch City Hotel loyalty programme             |
| Avg NPS Score     | 6.4        | 7.3          | Resort | Invest in City Hotel service training & upsells |
| Avg Satisfaction  | 3.3 / 5    | 3.7 / 5      | Resort | Improve checkin speed and food quality ratings  |

4.3 GitHub Repository & Collaboration

 **MILESTONE SCREENSHOT:** GitHub — Travel\_Data\_Dashboard repository (krimcy)

Screenshot at 3:32 PM shows the GitHub Settings > Collaborators page for the public repository krimcy/Travel\_Data\_Dashboard. The repo is public. A collaborator invite has been sent to 'Ishani Gurung' (hellcat101) with status 'Pending Invite — Awaiting hellcat101’s response'. The message 'User has already been invited' confirms a re-invite was attempted.

The project is version-controlled on GitHub at krimcy/Travel\_Data\_Dashboard as a public repository. Milestone screenshots are committed to the screenshots/ directory and tracked in git as required by the brief. A collaborator invitation has been sent to the project partner.

4.4 Network Infrastructure

 **MILESTONE SCREENSHOT:** macOS Internet Sharing — Wi-Fi shared via USB LAN

Screenshots at 5:45 PM and 5:53 PM show macOS System Settings with Internet Sharing ON, sharing the Wi-Fi connection (tuermereoam — Connected) to devices via USB 10/100/1000 LAN. This was used to provide internet access to a secondary machine during the project development session at SRH University, where available Wi-Fi networks included ESGast, ESint, KKI-Gast, and REMOTE26uwzv.

5. Challenges & Resolutions

| # | Challenge                                             | Root Cause                                                           | Resolution                                                                                 |
|---|-------------------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 2 | 23 tests SKIPPED (not failed)                         | hotel_bookings.csv not present; CSV files not yet generated          | Run generate_fake_data.py first; download Kaggle dataset to data/; tests then run and pass |
| 3 | Arrival date in 3 separate columns                    | Kaggle dataset splits year, month name, day into separate fields     | PySpark UDF maps month names to numbers; concat + to_date() builds proper DATE             |
| 4 | 31 Pylint warnings in test files                      | Type annotations, import order, docstring style                      | Non-blocking warnings; tests pass 15/15; Pylint score 8.75/10                              |
| 5 | Streamlit Basic Info showing 'None' for numeric stats | Describe() on object-typed columns returns None for mean/std/min/max | Expected behavior for VARCHAR columns; numeric columns display correctly                   |
| 6 | Collaborator invite 'already sent' message            | Re-invited same user (hellcat101) after first pending invite         | Informational message only; invite remains pending; no action needed                       |

## 6. Summary & Submission Checklist

| Requirement                  | Status     | Evidence                                                         |
|------------------------------|------------|------------------------------------------------------------------|
| Snowflake pipeline (1 of 2)  | ✓ Complete | sql/snowflake/*.sql; HOTEL_DB with RAW/CURATED/ANALYTICS schemas |
| 2 Business Objectives        | ✓ Complete | BO-1: Premium Revenue; BO-2: Traveler Satisfaction               |
| Data sources cited           | ✓ Complete | Antonio et al. (2019); Keatonballard Kaggle dataset              |
| No LLM-generated data        | ✓ Complete | generate_fake_data.py uses Python random(seed=42) only           |
| Spark data processing        | ✓ Complete | spark_transform.py (PySpark, Databricks-compatible)              |
| Dashboard reporting          | ✓ Complete | dashboard.html (3 tabs) + dashboard.py (Streamlit)               |
| Testing + downstream update  | ✓ Complete | 15 pipeline tests pass; TestDownstreamUpdate proves propagation  |
| JSON output                  | ✓ Complete | ANALYTICS.JSON_EXPORT via OBJECT_CONSTRUCT in SQL                |
| Milestone screenshots in git | ✓ Complete | screenshots/ directory committed to krimcy/Travel_Data_Dashboard |
| submission.txt               | ✓ Complete | Submission checklist in project root                             |
| Report (this document)       | ✓ Complete | 5-page formal report with evidence screenshots                   |

**Data Citation:** Antonio, A., de Almeida Rodrigues, A., & Nunes, L. (2019). Hotel booking demand datasets. *Data in Brief*, 22, 41–49. <https://doi.org/10.1016/j.dib.2018.11.126>

**Flight Data Citation: Keatonballard (2024). Synthetic Airline Passenger and Flight Data. Kaggle. <https://www.kaggle.com/datasets/keatonballard/synthetic-airline-passenger-and-flight-data>**

Upload CSV

synthet...r\_data.csv

1.4MB

+

Preview

|   | Passenger_ID | Flight_ID | Airline   | Departure_Airport | Arrival_Airport | Departure_Time      | Flight_Duration |
|---|--------------|-----------|-----------|-------------------|-----------------|---------------------|-----------------|
| 0 | P1           | F68       | Southwest | JFK               | ATL             | 2024-03-27 06:22:06 |                 |
| 1 | P2           | F1687     | Southwest | JFK               | DFW             | 2024-12-28 09:18:06 |                 |
| 2 | P3           | F1745     | United    | ORD               | ORD             | 2024-05-23 23:28:20 |                 |
| 3 | P4           | F1514     | Delta     | SEA               | ORD             | 2023-10-31 08:09:27 |                 |
| 4 | P5           | F1753     | Southwest | LAX               | JFK             | 2025-01-17 22:21:13 |                 |

Basic Info

|        | Passenger_ID | Flight_ID | Airline | Departure_Airport | Arrival_Airport | Departure_Time      | Flight_Duration |
|--------|--------------|-----------|---------|-------------------|-----------------|---------------------|-----------------|
| count  | 10000        | 10000     | 10000   | 10000             | 10000           | 10000               |                 |
| unique | 10000        | 1992      | 5       | 8                 | 8               | 9999                |                 |
| top    | P1           | F983      | America | LAX               | ORD             | 2023-07-24 01:04:00 |                 |
| freq   | 1            | 14        | 2065    | 1287              | 1282            | 2                   |                 |